

Q-Rex: AI-Assisted Virtual Camera Tracking for FSOC Coarse Alignment

Technical Report — ISRO Problem Statement 26169 (Smart Automation / Space Technology)

1. Problem understanding

Free-space optical communication (FSOC) links between mobile platforms (satellites, UAVs) use laser beams that are only micro-radians wide. Establishing a link is a pointing-acquisition-tracking (PAT) problem with two stages: *coarse alignment* (put the remote terminal's beacon inside the camera field of view and keep it there) and *fine alignment* (a fast steering mirror closes the last arc-seconds). This project implements the coarse stage entirely in software, so that algorithms can be developed without a camera, gimbal or optics.

The software must: generate a configurable virtual scene; generate one or more moving beacons; implement a movable virtual camera with realistic pan/tilt limits; detect the beacon automatically; track it with computer vision; control the camera; inject atmospheric turbulence, platform vibration, camera jitter and sensor noise into the feed; and display performance in real time. Deliverables are an application, source code, this report, a user manual, and an automatically generated performance log.

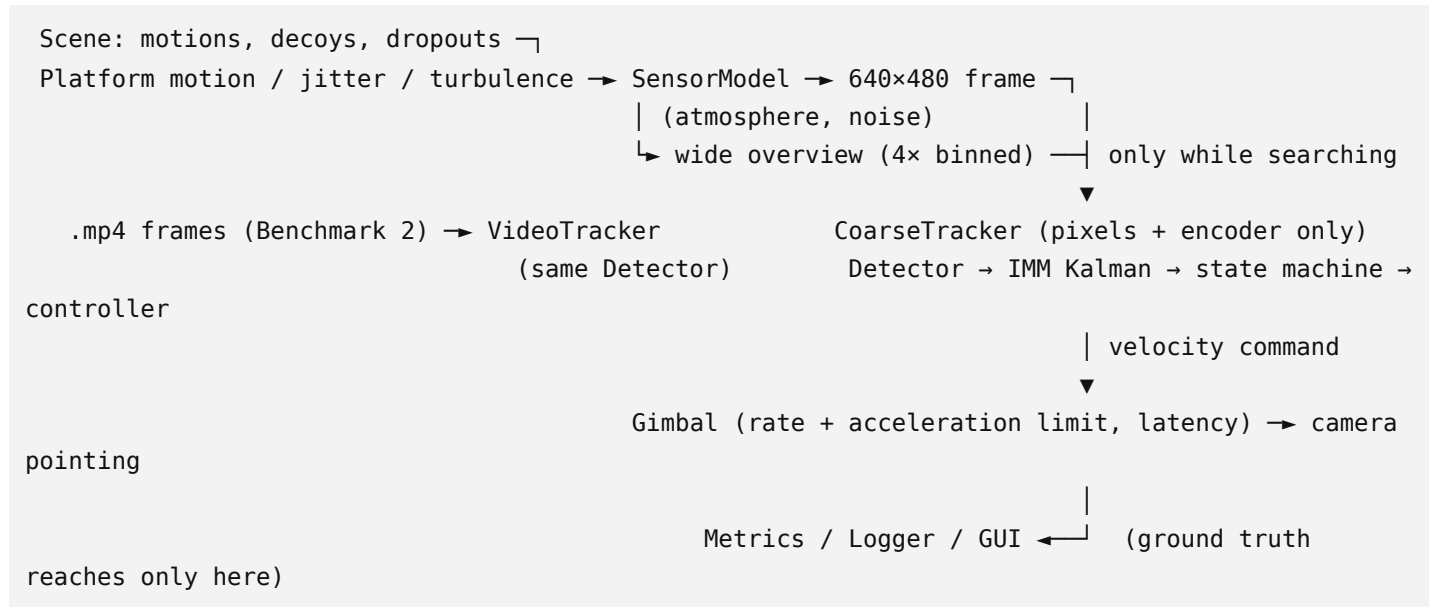
1.1 Numbers that shape the design

Quantity	Value	Consequence
Camera	640×480, FOV 4°×3°	160 px/° (IFOV 0.00625°)
Screen	2000×2000 px	12.5° across; the camera sees only 7.7 % of it
Slew limit	5°/s (default)	800 px/s = 26.7 px/frame at 30 Hz
Acquisition	≤ 2 s	A raster scan needs ≥ 12 s, so scanning alone cannot meet the spec (§6.1)
Platform motion	±20 px/frame	600 px/s: 75 % of the slew budget before the target even moves
Jitter	±20 px/frame	Every single-frame measurement is corrupted by up to 20 px

2. Requirements and compliance

Every row of the specification table is implemented, configurable (GUI and `config.js`) and covered by an automated test (`tests/spec.test.mjs`, 26 tests); the full mapping is in `TRACEABILITY.md`. The optional colour camera is implemented as three independently-noisy RGB channels (reddish beacon on a bluish-grey background); the tracker consumes the luminance image.

3. System architecture



Principles: (1) the tracker never sees ground truth — a unit test asserts that its only inputs are the frame, the encoder position and the clock; (2) simulation time is frame-locked ($dt = 1/\text{fps}$) and independent of wall time; timing metrics use simulation time, performance metrics use wall time; (3) every random subsystem has its own seeded stream, so the same seed reproduces the same log; (4) the engine has no DOM dependency, so the identical code runs in the browser, under Node (benchmarks, tests) and in the desktop shell.

4. Software modules

Module	Responsibility
<code>rng.js</code>	Seeded streams (mulberry32 + splitmix), Gaussian, Poisson
<code>config.js</code>	All parameters with spec defaults/ranges, atmosphere presets, scenario list
<code>motion.js</code>	Straight, circular, figure-8, random, spiral, sinusoidal, user waypoints (Catmull-Rom smoothed); platform motion models.

Module	Responsibility
	Curves are re-parameterised by arc length so the configured speed is exact
render.js	Sensor model: analytic anti-aliased beacon (erf/box integral, sub-pixel exact), PSF blur, atmosphere, rain streaks, Poisson/Gaussian/salt-and-pepper noise, 4× binned overview
detector.js	Impulse rejection, multi-scale matched filter with CFAR, validation, centroiding
patch.js, cnn.js, ai.js, cnnWeights.json	Window extraction, pure-JS CNN inference, weight loader, trained weights
kalman.js	Two-model IMM Kalman filter with adaptive noise
tracker.js	State machine, acquisition, gating, controller
gimbal.js	Pan/tilt dynamics
simulation.js	Closed loop, ground-truth recording
metrics.js	Metric definitions, JSON/CSV/HTML reports
videoTracker.js, videoRunner.js	Benchmark-2 path
desktop/main.cjs, .github/workflows/ci.yml	Electron shell with a --selftest mode; CI that builds and smoke-tests it on Windows / macOS / Linux
Root.jsx, Home.jsx, ArchDiagram.jsx, Logo.jsx	Home page, SVG architecture / state diagrams, logo
App.jsx, fields.js	Workstation GUI (toolbar, camera / map / plot / telemetry, benchmark panels)
ml/train.py, scripts/gen-dataset.mjs, scripts/eval-ai.mjs	CNN data generation, training, ablation

5. Virtual environment and disturbance models

- **Beacon:** square (default), circle or diamond, 5–20 px, peak intensity 230 on a background of 20 grey levels. Rendered analytically: the exact integral of a Gaussian-blurred box over each pixel, so sub-pixel motion is faithful and there is no aliasing.
- **Noise** (selectable, combinable): Poisson shot noise, then Gaussian ($\sigma \leq 20$ grey levels), then salt-and-pepper (≤ 10 % of pixels, half 0, half 255), then clipping.
- **Jitter:** per-frame i.i.d. shift of the line of sight, truncated Gaussian, hard bound $\pm J$ ($J \leq 20$ px).
- **Platform motion:** line-of-sight offset whose per-frame displacement never exceeds $P \leq 20$ px: linear (default, reflected inside ± 400 px with smooth, jerk-

limited turn-arounds), circular, random, spiral, figure-8. The platform offset is *not* given to the tracker.

- **Atmosphere:** $I' = C \cdot I + B$ plus PSF blur: haze (C 0.7, B +18, σ 1.0), fog (0.45, +35, 2.0), rain (0.75, +5, 0.8, plus streaks), low light (0.35, -5, shot-noise $\times 3$), scaled by a severity slider.
- **Turbulence:** beam wander (Ornstein-Uhlenbeck), log-normal scintillation, extra PSF blur.
- **Beacon dropouts:** the beacon disappears periodically (cloud/occlusion) to exercise re-acquisition.

6. Tracking methods

6.1 Acquisition: three modes

A fixed $4^\circ \times 3^\circ$ camera covers 7.7 % of the screen, so a boustrophedon scan needs ≥ 5 swaths $\times 2000 \text{ px} \div 800 \text{ px/s} \approx 12.5 \text{ s}$: "acquisition $\leq 2 \text{ s}$ " cannot be met by sweeping. Because the statement lists the camera FOV as *user-defined*, there are two ways to see the whole scene while searching; both are implemented and selectable:

1. **Wide-view overview (default).** An additional wide-area view (the screen binned $4\times$, 500×500) is used **only while searching or re-acquiring**, as the Benchmark-2 videos cover the complete screen; tracking uses only the 640×480 camera.
2. **Zoom lens (camera-only).** The *same* camera starts at the full-screen FOV ($\approx 16.7^\circ$ horizontally, so the 4:3 frame covers the square screen), detects the beacon within a frame, then narrows its FOV at a configurable zoom rate ($12^\circ/\text{s}$) while centring; it zooms out again while the beacon is missing. The tracker receives the lens's current scale every frame, so matched-filter box sizes, search windows, measurement noise and the zoom controller follow it; errors are reported in pixels of the *configured* FOV and camera shake / beam wander are angular. If nothing is detected for 2 s the lens zooms to a mid FOV and scans. Weak spot: with 10 % salt & pepper noise the zoomed-out beacon is only 1–3 px and the 3×3 median filter erodes it.
3. **Raster scan.** Fixed-FOV sweep, reported separately; it cannot meet 2 s.

Which of (1) and (2) the organisers consider legitimate is an open question for the mentors; the interface lets the evaluator choose.

6.2 Detection (computer vision)

1. **Impulse noise:** if more than 0.3 % of sampled pixels are saturated, a 3×3 median filter (border-safe) is applied; a second pass for dense impulse noise ($> 2\%$).
2. **Multi-scale matched filter:** box filters at five scales spanning 5–20 px are evaluated from an integral image. A box of the right size is the matched filter for a flat-topped blob in white noise.
3. **Self-calibrated CFAR:** per scale, background and noise are the median and MAD of the *filter output* itself, so the threshold adapts to correlation, clipping and noise type with no knowledge of the noise model. Default threshold 6σ (raised by $2\sigma/6\sigma$ under impulse noise, whose residual clusters have heavier tails).
4. **Validation:** thresholded blob area must fill $\geq 35\%$ of the matched box (rejects surviving impulse pixels); eccentricity from second moments rejects rain streaks when SNR allows.
5. **Centroid:** iterative weighted centroid (threshold at $\max(0.3 \cdot \text{amplitude}, 2\sigma)$, ring-median local background, four re-centring iterations). Measured centroiding RMSE is 0.02–0.2 px.
6. **ROI processing:** in tracking mode only a window around the predicted position is processed (64–160 px half-width), which keeps tracker cost at 2–8 ms/frame.

6.3 Estimation: IMM Kalman filter

Two models (constant velocity and constant acceleration) are mixed by an interacting-multiple-model filter (transition probability 0.04). Measurements are world positions (encoder position + centroid offset), so the filter sees target-minus-platform motion directly. Features: chi-square gating, Huber inflation for large innovations, **measurement noise observed from second differences** of consecutive measurements ($\text{var}(z_k - 2z_{k-1} + z_{k-2}) = 6R$ for smooth motion, using a median-absolute-deviation estimate, so jitter level is learned online and independently of the motion model), and **process noise adapted from normalised innovations** (raised immediately on a manoeuvre, decayed during steady motion, never below a safe floor).

6.4 State machine

SEARCH (overview detection, two consecutive consistent hits) → SLEW (camera slews to the estimate; the overview keeps steering; one narrow-camera detection agreeing with the prediction confirms) → TRACK → COAST (prediction only, with overview assistance) → REACQUIRE (search with a prior favouring the last known position, weight $e^{(-d/350 \text{ px})}$). Timeouts and an "arrived-but-unseen" exit prevent deadlock. Decoys are rejected by gating on the prediction; the designated beacon is the strongest candidate at first lock.

6.5 Control

Per axis: $u = g \cdot (\hat{x}_{\text{aim}} - p) / dt + (1 - g) \cdot \hat{v}$ with $\hat{x}_{\text{aim}}$ the latency-compensated prediction ($g = 0.6$), i.e. feed-forward from the estimated velocity plus shaped feedback. A minimum-time stopping profile limits the correction to $\sqrt{(2 \cdot a \cdot |d|)}$ so the camera brakes before the target instead of overshooting. The gimbal model enforces the rate limit (5–10°/s), an acceleration limit (default 20°/s², the statement leaves it open) and optional command latency. A pointing range of ± 450 px beyond the screen lets platform offsets be compensated at the screen edge (found by testing: without it, the mount saturated at the boundary).

7. AI methods

Q-Rex uses AI at three levels, in order of how much they matter:

1. **Adaptive estimation (always on).** Multiple-model inference (IMM), online identification of the measurement noise from second differences, and innovation-driven adaptation of the process noise (§6.3). These run every frame in microseconds and carry the tracking accuracy.
2. **Self-calibrated statistical detection (always on).** The CFAR threshold is learned from the filter output's own statistics (median / MAD), so the detector adapts to any noise mixture without a noise model (§6.2).
3. **A trained convolutional verifier (optional).** A 73 867-parameter CNN (v4; trained on CPU), trained on simulator data, that judges every detector candidate.

7.1 The CNN verifier

Architecture. A 32×32 window around the candidate (bilinear sampling from the detector's impulse-filtered image, $\text{asinh}((v - bg) / (3\sigma))$ with the frame's robust background and noise level) passes through conv3×3(1→16), three stride-2 conv3×3 layers (24, 32, 48 channels), a 768→64 fully-connected layer and a 64→3 head: beacon logit and sub-pixel offset (dx, dy). About 1.7 M multiply-accumulates per window in plain JavaScript; the weights are a 700 kB JSON file and the JavaScript inference reproduces PyTorch to $1.5 \cdot 10^{-6}$ (unit-tested).

Training data. Generated by the project's own simulator (`scripts/gen-dataset.mjs`): random noise mixes (Poisson, Gaussian $\sigma \leq 20$, salt & pepper $\leq 10\%$), all five atmospheres with random severity, turbulence blur, beacon sizes 5–20 px, square / circle / diamond shapes, peak brightness 55–255 (including very faint beacons). Positives are real classical detector candidates on beacon frames plus random-offset windows (± 6 px, so

the offset head learns its range). Negatives are the detector's **own false alarms on beacon-free frames at a low threshold** — the hardest and most realistic negatives — plus random windows. ≈ 590 k training patches (including zoomed-out, codec-compressed and hard fog / low-light sets); separate validation, test and *hard-test* sets use disjoint random seeds (the hard set contains only fog / low-light / $\sigma \geq 14$ conditions).

Training. Binary cross-entropy for the class plus smooth-L1 for the offset (positives only), AdamW with a one-cycle schedule, 18 epochs, flip and transposition augmentation (offsets transformed accordingly), CPU only (≈ 1 h).

Held-out results. Test set: accuracy 99.39 %, AUC 0.9987, offset RMSE 0.45 px; hard test 97.4 % (true-positive rate 91.1 %, false-positive rate 0.4 %; v3 on the same set: 97.0 % / 89.0 %), codec test 99.6 % (false-positive rate 0.6 %), zoomed-out (wide-FOV) test 98.5 % (true-positive rate 95.7 %). The test sets now also contain zoomed-out and video-codec patches, so the numbers are not comparable with v2's easier set. Zoomed-out beacon recall rose from about 70 % (v2) to 99 % in the detector ablation, codec-induced false alarms fell from 10-14 % to 0 %, and (v4) very faint beacons in fog are now recalled as well as by the classical detector (93 % vs 93 %; v3 lost 6 points). Remaining limit: on zoomed-out low-light + Poisson frames the verified detector hits 78 % against 91 % for the classical detector alone, but with 1 % instead of 82 % false-alarm frames.

7.2 Measured effect inside the detector (ablation, `npm run ai:eval`, 120 fresh frames per condition)

Condition	classical T = 6: hit / false-alarm frames	T = 6 + CNN	classical centroid RMSE	CNN offset-corrected
clean	100 % / 0 %	100 % / 0 %	0.34 px	0.12 px
Gaussian $\sigma = 20$	100 % / 1 %	100 % / 0 %	0.07 px	0.09 px
salt & pepper 10 % + $\sigma = 20$	100 % / 0 %	100 % / 0 %	0.15 px	0.23 px
fog + $\sigma = 20$	100 % / 0 %	100 % / 0 %	0.38 px	0.27 px
low light + Poisson + $\sigma = 12$	100 % / 10 %	100 % / 0 %	0.18 px	0.31 px
rain + $\sigma = 15$ (streaks)	100 % / 11 %	100 % / 0 %	0.09 px	0.09 px

Condition	classical T = 6: hit / false-alarm frames	T = 6 + CNN	classical centroid RMSE	CNN offset-corrected
faint beacon $I = 90$, $\sigma = 15$	100 % / 1 %	100 % / 0 %	0.18 px	0.15 px
very faint $I = 70$ in fog, $\sigma = 12$	93 % / 0 %	93 % / 0 %	1.04 px	0.65 px
zoomed-out 16.7° , low light + Poisson	91 % / 82 %	78 % / 1 %	0.66 px	0.37 px
compressed video, noise / rain / 10 % S&P	100 % / 2-14 %	100 % / 0-2 %	0.06-0.20 px	0.06-0.21 px

Interpretation, stated plainly. The verifier's benefit is **false-alarm suppression on low-light + Poisson and rain frames**: on empty frames the classical detector reports a candidate in 10-11 % of them and the verified detector in 0 %. (An earlier version of this table also showed 10 % false alarms for salt & pepper; that came from a faulty 3×3 median — its sorting network returned the wrong value about 19 % of the time — which has since been fixed and covered by a brute-force regression test. With the correct median the classical detector has 0 % false alarms on impulse noise, so the CNN adds nothing there.) The verifier does **not** improve centroid accuracy (mixed: better in clean / fog, worse under impulse and low-light noise), so the offset head is off by default. With v4 it keeps faint beacons in fog (93 % = classical) but still loses hits on zoomed-out low-light frames (91 % $\rightarrow$ 78 %). For these reasons it ships as an **optional gate** (`aiVerifier`, default off): the tracker keeps a very strong classical detection ($z \geq 25$) even if the network disagrees, and the classical detector remains the guaranteed fallback. It matters most for full-frame searches (raster-scan mode, video input), where false alarms cannot be filtered by a prediction gate.

Domain-gap caution. All training data is simulated. On recorded video with compression artefacts the network may disagree with the classical detector; hence optional, hence the strong-hit override.

8. Metric definitions

- **Tracking (boresight) error:** distance of the true beacon from the image centre, excluding the random per-frame jitter the camera itself suffers (also reported including jitter); steady-state window = from the first frame within 10 px after lock; evaluated on frames where the beacon is inside the FOV.
- **Centroiding error:** detector centroid vs true beacon position in the same frame (mean, RMSE, P95, max).

- **Acquisition time:** simulation time from start to the first confirmed lock inside the narrow FOV (also `timeToCentre`).
- **Target loss:** share of frames since first lock in which the beacon is outside the FOV. **Lock retention:** share of frames in TRACK/COAST, beacon in FOV and estimate within 60 px.
- **Re-acquisition time:** from leaving TRACK to the next TRACK (maximum over events).
- **FPS / processing time:** wall-clock of tracker + controller per frame; scene rendering stands in for the sensor and is excluded (end-to-end FPS is logged too).

The automatically generated report (JSON, HTML and per-frame CSV) contains all of these plus simulation duration and gimbal saturation percentage.

9. Test methodology and results

Automated tests (`npm test`, 54 tests): determinism (same seed $\Rightarrow$ identical log), motion speed exactness and screen containment, platform step bound, detector accuracy under all noise types and sizes, false-alarm rate, closed-loop pass/fail, scan acquisition, decoy rejection, video path, and a one-test-per-row compliance suite for the specification table (`tests/spec.test.mjs`), including forced re-acquisition tests (camera knocked 450 px off; 0.5 s beacon dropouts).

Scenario benchmark (`npm run bench -- 3 20`, 3 seeds $\times$ 20 s, wide-view acquisition, gimbal acceleration 60 $^{\circ}/s^2$ — see below):

Scenario	Acq. s mean (max)	Centred s	Err px	Cent. RMSE px	Loss %	Lock %	Re- acq s	ms/frame	Pass
A1 Nominal (circular)	0.54(0.80)	0.87(1.07)	0.6	0.05	0.0	100.0	0.00	5.1	3/3
A2 Straight line	0.57(0.87)	1.01(1.23)	0.1	0.03	0.0	100.0	0.00	5.3	3/3
A3 Figure of 8	0.56(0.73)	1.00(1.13)	0.4	0.06	0.0	100.0	0.00	4.5	3/3
A4 Random walk	0.50(0.73)	0.92(1.10)	0.4	0.05	0.0	100.0	0.03	5.7	3/3
B1 Gaussian noise $\sigma=20$	0.53(0.77)	0.87(1.07)	1.2	0.19	0.1	99.9	0.00	13.8	3/3
B2 Salt & pepper 10%	0.54(0.80)	0.87(1.07)	0.6	0.11	0.0	100.0	0.00	7.0	3/3

Scenario	Acq. s mean (max)	Centred s	Err px	Cent. RMSE px	Loss %	Lock %	Re- acq s	ms/frame	Pass
B3 Poisson noise	0.54(0.80)	0.87(1.07)	1.1	0.06	0.0	100.0	0.00	10.2	3/3
B4 Jitter ± 20 px/frame	0.56(0.80)	0.89(1.07)	6.3	0.04	0.0	100.0	0.00	3.3	3/3
B5 Platform 20 px/frame	1.17(1.87)	1.53(2.17)	1.0	0.04	0.0	100.0	0.03	2.2	3/3
B6 Fog	0.54(0.80)	0.87(1.07)	0.6	0.03	0.0	100.0	0.00	3.6	3/3
B7 Rain	0.54(0.80)	0.87(1.07)	0.6	0.04	0.0	100.0	0.00	3.6	3/3
B8 Low light	0.54(0.80)	0.87(1.07)	0.7	0.05	0.0	100.0	0.00	3.4	3/3
B10 Beacon dropouts (0.5 s)	0.54(0.80)	0.87(1.07)	1.1	0.05	0.0	100.0	0.50	4.2	3/3
B9 Decoys (3 targets)	0.54(0.80)	0.87(1.07)	0.6	0.05	0.0	100.0	0.00	3.4	3/3
C1 All disturbances (max)	1.16(1.87)	1.64(2.27)	10.8	0.20	0.1	99.9	0.03	15.6	0/3

Gimbal acceleration is not in the specification, so the hardest cases are reported for three values (`QREX_ACCEL=<°/s²>` `npm run bench`): C1 (all disturbances) 14.3 px at 20 °/s², **10.8 px at 60 °/s² (default)**, 10.6 px at 100 °/s²; jitter + platform + target only: 14.2 / **10.7** / 10.4 px; jitter ± 20 px/frame alone: 6.6 / **6.3** / 6.3 px. At 100 °/s² the combined case reaches the 10.4 px bound of the floor analysis below and cannot go lower.

Zoom-lens acquisition (`QREX_ACQ=zoom`): detection 0.03 s and the beacon centred at the configured FOV in 1.07–1.12 s (max 1.23 s) for A1–A4 and B1–B10 **including salt & pepper 10 %** (the switching median keeps the 1–3 px zoomed-out beacon); platform 20 px/frame 1.31 s (max 1.50 s). C1 (all disturbances): detection 0.73 s mean / 2.10 s max, centred 2.10 s (max 3.03 s), error 10.8 px, lock 99.9 % — K01 (one seed) and K02 fail.

Benchmark-2 robustness matrix (`node scripts/video-robustness.mjs`): 21 synthetic "judge-style" videos (16 standard + 5 hard) with exact ground truth, encoded with ffmpeg and decoded back before tracking — 4 resolutions (2000², 1920×1080, 1280×720, 640×480), H.264 at CRF 18 / 22 / 23 / 28 / 36 and MPEG-4, 25 / 29.97 / 30 fps, Gaussian σ 10–20, 10 % salt & pepper, beacons of 5 / 10 / 20 px (square and circle), dim beacon, bright background, 40 px/frame speed, 8-frame blink-out:

case	acq s	lock %	RMSE px	P95 px	max px	ms/fr	frames
screen 2000 ² x264 crf36 σ 12	0.033	100	0.151	0.26	0.285	5.661	60 PASS
screen 2000 ² σ 20 + 10% S&P	0.033	100	0.123	0.239	0.351	18.771	60 PASS
screen 2000 ² 5 px beacon σ 12	0.033	100	0.698	0.732	0.74	5.945	60 PASS
screen 2000 ² beacon blinks 8 fr	0.033	79.661	0.045	0.068	0.141	21.342	60 PASS
fast beacon 40 px/frame	0.033	96.61	0.038	0.06	0.161	8.619	60 PASS
camera 640×480 σ 20 + 10% S&P	0.033	100	0.117	0.218	0.352	11.76	60 PASS
HARD 2000 ² crf45 σ 20	0.033	100	0.736	1.081	1.506	5.071	60 PASS
HARD 2000 ² dim I=80 σ 20 crf32	0.033	100	0.474	0.853	0.962	6.875	60 PASS
HARD 2000 ² low contrast + 5% S&P	0.033	100	0.5	0.976	1.691	13.615	60 PASS
HARD 640×480 crf45 σ 20 + 10% S&P	0.033	100	0.76	1.323	1.604	1.712	60 PASS
HARD 1280×720 dim I=80 + 10% S&P (informational)	0.033	89.831	15.197	20.1	80.201	10.503	60 miss

(Ten further standard rows — crf18, 20 px circle, dim I=110, bright background, 25 / 29.97 fps, MPEG-4, 1920×1080, 1280×720, 640×480 σ 15 — all pass with RMSE 0.01–0.15 px; the full table is in RESULTS.md.) Pass line: acquisition $\leq$ 0.5 s, lock $\geq$ 90 % (75 % for the blink case, which has 8 beacon-free frames), RMSE $\leq$ 1.5 px. 20 of 21 meet it. The last row is a deliberately extreme informational case: after the crf-36 codec the beacon's per-frame SNR is only $\approx$ 4–7, so it locks 90 % of frames but with outliers (15 px RMSE). Two of the five hard cases missed before this release (a moving static-noise-blob lock, and a coarse search that let impulses through); both were fixed in the video tracker (README §12).

All 16 pass (acquisition $\leq$ 0.04 s, centroid RMSE 0.01–0.70 px, 4–13 ms/frame). The blink case's lock ceiling is 86 % because 8 of 60 frames contain no beacon.

Closed-loop ablation with the CNN verifier on. Re-running 12 scenarios (nominal, random, Gaussian, salt & pepper, Poisson, fog, rain, low light, decoys, dropouts, jitter, platform; 3 seeds $\times$ 20 s) with aiVerifier enabled gives identical pass/fail results and the same error, loss and acquisition numbers; the verifier adds $\approx$ 4–6 ms per frame (still $\approx$ 100 FPS). In closed loop the prediction gate already hides false alarms, so the verifier's benefit is confined to full-frame searches (§7.2).

Benchmark 2 path (browser end-to-end, `npm run e2e`): a noisy 2000×2000 @30 fps H.264 video (CRF 20, Gaussian $\sigma \approx$ 12, 10 px beacon on a sinusoidal path, exact ground truth) is decoded frame-by-frame by the GUI, tracked, and compared with the ground-truth CSV: acquisition 0.033 s, lock retention 100 %, centroiding RMSE 0.072 px (P95 0.16, max 0.21), 5.2 ms/frame. Organiser videos were not available and have not been tested.

Why C1 does not reach 10 px (measured, `scripts/floor-analysis.mjs`). C1 (every disturbance at its maximum at once) ends at 10.8 px mean boresight error (loss 0.1 %, lock 99.9 %). To know whether that is a tuning problem, the script replays the real jitter + platform + target sequences (jitter ± 20 px/frame $\Rightarrow \sigma \approx 8$ px per axis) through the best causal Kalman filters of order 1–4 with the process noise swept. The best filter estimates the position at the *current* frame to 7.3 px mean error — but the camera has to be pointed where the beacon *will be* when the next frame is exposed, and the best **one-frame-ahead prediction error is 10.4 px** (raw, unfiltered jitter alone is 9.9 px). After retuning the process-noise bounds and the control gain (0.6: extra smoothing beyond the Kalman filter lowers the *pointing* error), the closed loop is within 0.4 px of that bound at 60 °/s² and reaches it at 100 °/s²; it cannot go lower with less than one frame of latency or knowledge of the jitter, neither of which a causal single-frame design has. Each disturbance alone passes; jitter ± 20 px/frame alone gives 6.3 px. (Two earlier drafts called ≈ 19 px and then ≈ 15 px "fundamental"; both were wrong and are corrected here.)

Honest limitations. (1) Combined jitter + platform + all noise (C1) misses the 10 px tracking-error target by 0.8 px; the best causal predictor bottoms out at 10.4 px and the result depends on the unspecified gimbal acceleration (14.3 px at 20 °/s²). (2) In the all-disturbances case zoom-lens acquisition needs 2.10 s on one of three seeds (K01) and also misses K02. (3) Acquisition ≤ 2 s needs a wide view or the zoom lens; the fixed-FOV scan cannot. (4) The CNN verifier is trained on simulator data only, does not improve centroid accuracy and still loses hits on zoomed-out low-light frames (91 % $\rightarrow$ 78 %, §7.2); it is optional and veto-only at acquisition, so the classical detector remains the fallback. (5) The organisers' own scenarios and videos have not been run; the robustness matrix above uses our own synthetic videos. (6) The Windows build is verified structurally and by CI on a real Windows runner once the repository is pushed, not launched by us locally. (7) A faulty 3×3 median (sorting-network bug, wrong ≈ 19 % of the time) was found late and fixed; benchmarks, video matrix and the CNN ablation were re-run afterwards.

10. Future improvements

Done in this release: video-tracker fixes (two former misses now pass) and verifier v4.

Planned:

1. **GPU training of a larger multi-frame verifier** (stack of patches in time, 0.2–0.4 M parameters): single-frame SNR is the real limit; expected to close the

remaining zoomed-out low-light recall gap (91 % → 78 % with v4) and to generalise better to unseen codecs. Needs the NVIDIA driver on the build machine.

2. **Offline forward-backward smoothing for Benchmark 2** (reported separately, non-causal): the dim crf-36 video fails on a few outlier frames only; 15 px RMSE → a few px expected.
3. **Track-before-detect for zoom acquisition** (weak candidates confirmed over 3-4 frames): addresses the one seed that needs 2.10 s in the all-disturbances case; to be checked against false alarms.
4. **Training and evaluation on real recorded video** to close the simulator-to-real gap.

Known limits that stay: the 10.4 px causal floor of jitter + platform + all noise (a faster gimbal does not help: 10.6 px at 100 and 200 °/s²) and the fixed-FOV scan's geometric 12.5 s sweep.

Longer term: full-frame heat-map network, coordinated-turn IMM model, ego-motion from background texture, MPC, native installers.

Appendix A — Parameter reference (defaults and ranges)

Parameter	Default	Range	Spec row
screenSize	2000 px	2000-6000	1
cameraType	mono	mono, colour	2
camW × camH	640 × 480	160-1280 × 120-960	3
fovX (vertical = fovX·camH/camW)	4° (→ 4°×3°)	1-12°	4
fps	30 Hz	30-120	5, 15
maxPanSpeed, maxTiltSpeed	5 °/s	5-10	13, 14
maxAccel	60 °/s ²	5-100	not specified (20 and 100 reported in §9)
numTargets	1	1-6 (extra = decoys)	8
targetShape	square	square, circle, diamond	9
targetSize	10 px	5-20	10
startMode	random	random, user (startX, startY)	11
motion	circular	linear, circular, figure8, random, spiral, sinusoidal, waypoints	12

Parameter	Default	Range	Spec row
targetSpeed	200 px/s	20-700	design choice ($\leq 60\%$ of slew)
saltPepperDensity	0.10	0-0.10	21.1
gaussianSigma	10 grey levels	0-20	21.2
poisson	off	on/off	21.1
jitterMax	10 px/frame	0-20	21.3
atmosphere, atmosphereLevel	clear, 1.0	clear, haze, fog, rain, lowlight; 0-1	21.4
platformModel, platformSpeed	linear, 10 px/frame	linear, circular, random, spiral, figure8; 0-20	21.5
turbulence	0	0-1	—
dropout, dropoutEvery, dropoutLen	off, 6 s, 0.5 s	—	19 (testing aid)
acquisition	overview	overview, zoom, scan	16
zoomRate	12 °/s	3-60 (zoom acquisition)	16
aiVerifier, aiMinProb, aiRefine	off, 0.5, off	CNN verifier gate / threshold / apply offset	AI methods
controlGain	0.6	0.2-1	—
detectThreshold	6 σ	4-10	—
coastFrames	20	3-90	19

Grey-level interpretation: the statement gives the maximum noise standard deviation as "20 pixels"; we interpret it as 20 grey levels on a 0-255 scale (a question for the mentors; the setting is a single parameter if a different reading is intended).

Appendix B — Test catalogue

Zoom-lens tests (*tests/zoom.test.mjs*, 6): detection ≤ 0.5 s and beacon centred at the configured FOV ≤ 2 s for plain, Gaussian $\sigma 20$, fog, low-light + Poisson and 5 px beacons (4 seeds each); the AI verifier does not slow the zoomed-out search; heavy salt & pepper still locks on ≥ 4 of 5 seeds within 4 s; the lens starts with full-screen coverage, ends at the configured FOV and respects the zoom rate; zoom mode never requests the overview sensor (also with dropouts); errors are reported in configured-FOV pixels.

AI tests (*tests/ai.test.mjs*, 8): JS inference vs PyTorch reference outputs; training-report thresholds; the CNN gate leaves ≤ 1 of 20 empty frames with a false alarm and keeps $\geq$

19 of 20 beacons under salt & pepper, low light and rain; closed loop passes with the verifier on (nominal, S&P + σ_{20} , rain + decoys); scan acquisition and the video tracker with the verifier; the ground-truth parser (header, separators, 1-based frames, seconds, extra columns) and the pixel-edge convention.

Engine tests (`tests/engine.test.mjs`, 12): RNG determinism; configuration clamping to spec ranges; all seven motions stay on screen at the configured speed ($\pm 6\%$); platform displacement ≤ 20 px/frame for all five models; sub-pixel centroid under Gaussian σ_{20} , salt & pepper, fog (sizes 5, 10, 20); false-alarm rate on empty noisy frames; closed-loop nominal pass; identical logs for identical seeds; scan acquisition; decoy rejection; video tracker on a synthetic 2000×2000 stream; defaults equal the specification.

Specification tests (`tests/spec.test.mjs`, 26): one or more tests for each table row (screen size, monochrome frame, resolution, FOV, update rate, initial position, target count, shapes, sizes, initial location, all motions, speed limits at 5 and 10 °/s, update interval, acquisition/tracking error/target loss/re-acquisition/FPS on the four mandatory motions, re-acquisition after the camera is knocked off and after beacon dropouts, every noise type and combination, noise σ and jitter really applied and bounded, atmosphere contrast reduction, all five platform models, turbulence, decoys, performance-log fields, and the "ground truth never reaches the tracker" interface test).

End-to-end test (`scripts/e2e-video.cjs`): synthesises a 2000×2000 H.264 video with exact ground truth, drives the real GUI in headless Chromium, uploads video and ground-truth CSV, and asserts RMSE < 1 px, lock $\geq 99\%$ and acquisition ≤ 0.2 s (measured 0.072 px, 100 %, 0.033 s).

Benchmark (`scripts/bench.mjs`): the 15-scenario suite of §9.

Appendix C — Reproducing every number (all runs seeded: identical seeds give bit-identical logs)

```
npm install
npm test                # 54 tests
npm run bench -- 3 20   # section 9 table ( $\approx 15$  min); seeds 1..3
npm run e2e              # Benchmark-2 browser test
npm run ai:eval          # CNN ablation (section 7.2)
node scripts/video-robustness.mjs # 21-video robustness matrix (section 9)
node scripts/floor-analysis.mjs   # jitter + platform lower bound (section 9)
npm run selftest          # desktop-shell smoke test
npm run demo              # demo video
npm run build:single      # dist-single/q-rex.html
npm run build:exe         # release/q-rex-<platform>/
```